

Implementasi Dinamis Client Side Dalam Aplikasi Manajemen Kost

Client-side rendering digunakan untuk komponen yang membutuhkan interaktivitas tinggi, state management, dan real-time updates.

1. Real-time Notification System

File: src/app/dashboard-user/page.tsx

Implementasi:

```
useEffect(() => {
  const fetchData = async () => {
    // Fetch initial data
    // ...
  };

  fetchData();

  // Setup Supabase Realtime Channel
  const channel = supabase
    .channel("booking_requests-changes")
    .on(
      "postgres_changes",
      {
        event: "*",
        schema: "public",
        table: "booking_requests",
      },
      () => {
        fetchData(); // Re-fetch saat ada perubahan
      }
    )
    .subscribe();

  return () => {
    supabase.removeChannel(channel);
  };
}, [router, supabase, selectedKost]);
```

Penjelasan:

- Menggunakan **Supabase Realtime** untuk listen perubahan di tabel `booking_requests`
- Saat admin approve/reject booking, user langsung mendapat update tanpa refresh
- Notification badge di bell icon update otomatis
- Cleanup channel saat component unmount untuk menghindari memory leak

Dynamic Behavior:

- Bell icon badge menampilkan jumlah booking dengan status 'process'
 - Angka berubah secara real-time saat admin proses booking
 - Jika >9, tampilkan "9+"
-

2. Countdown Timer untuk Payment

File: src/app/dashboard-user/components/CountdownTimer.tsx

Implementasi:

```
export default function CountdownTimer({ expiresAt, onExpire }:
CountdownTimerProps) {
  const [timeLeft, setTimeLeft] = useState<number>(0);

  useEffect(() => {
    const calculateTimeLeft = () => {
      const difference = new Date(expiresAt).getTime() - new
Date().getTime();
      return Math.max(0, difference);
    };

    setTimeLeft(calculateTimeLeft());

    const timer = setInterval(() => {
      const newTimeLeft = calculateTimeLeft();
      setTimeLeft(newTimeLeft);

      if (newTimeLeft === 0) {
        clearInterval(timer);
        onExpire(); // Trigger callback saat waktu habis
      }
    }, 1000);

    return () => clearInterval(timer);
  }, [expiresAt, onExpire]);

  const minutes = Math.floor(timeLeft / 60000);
  const seconds = Math.floor((timeLeft % 60000) / 1000);
  const isUrgent = minutes < 3;

  return (
    <div className={`flex items-center gap-2 px-4 py-2 rounded-lg ${
      isUrgent ? 'bg-red-100 text-red-700' : 'bg-yellow-100 text-yellow-700'
    }`} >
      <Clock className="w-4 h-4" />
      <span className="font-semibold text-sm">
        {timeLeft === 0 ? 'Waktu Habis' : (
          <>Sisa Waktu: {minutes.toString().padStart(2,
'0')}:{seconds.toString().padStart(2, '0')}</>
        )}
      </span>
    </div>
  );
}
```

Penjelasan:

- Timer countdown real-time dari `booking_expires_at` (10 menit)
- Update setiap 1 detik menggunakan `setInterval`
- **Dynamic color:** Red jika <3 menit (urgent), Yellow jika ≥ 3 menit
- Automatic callback saat timer habis
- Format: MM:SS dengan leading zeros

Dynamic Behavior:

- Warna background berubah dari yellow ke red saat <3 menit
 - Text berubah dari countdown ke "Waktu Habis" saat expired
 - Component otomatis cleanup interval saat unmount
-

3. Multi-step Payment Modal

File: src/app/dashboard-user/components/PaymentModal.tsx

Implementasi:

```
export default function PaymentModal({ open, onClose, bookingId }:
PaymentModalProps) {
  const [step, setStep] = useState(1); // Track current step
  const [method, setMethod] = useState<"rekening" | "qris" | null>(null);
  const [file, setFile] = useState<File | null>(null);
  const [showSuccess, setShowSuccess] = useState(false);
  const [showError, setShowError] = useState(false);

  // Fetch payment methods saat modal open
  useEffect(() => {
    if (!open) return;
    const fetchMethods = async () => {
      const { data } = await
supabase.from("payment_methods").select("*");
      setMethodsData(data);
    };
    fetchMethods();
  }, [open, supabase]);

  // Dynamic rendering based on step
  return (
    <div className="fixed inset-0 z-50">
      { /* Progress Indicator - Dynamic color based on step */ }
      <div className="flex items-center justify-between mb-6">
        {[1, 2, 3, 4].map((s) => (
          <div className={`w-8 h-8 rounded-full ${
            (step >= s || showSuccess)
              ? "bg-emerald-600 text-white"
              : showError && s === 4
              ? "bg-red-600 text-white"
              : "bg-gray-200 text-gray-500"
          }`} >
            {s}
          </div>
        ))}
      </div>

      { /* Step 1: Pilih Metode */ }
      {step === 1 && (
        <div>
          {methodsData.map((m) => (
            <button onClick={() => {
              setMethod(m.type);
              setStep(2); // Navigate ke step 2
            }}>

```

```

    }}>
    {m.type === "rekening" ? `Transfer
    ${m.bank_name}` : "QRIS"}
    </button>
    )}}
  </div>
)}

{/* Step 2: Display Payment Info - Dynamic based on method */}
{step === 2 && method === "rekening" && (
  <div>
    <p>No Rek: {selectedMethod.account_number}</p>
    <button onClick={() => setStep(3)}>Lanjut</button>
  </div>
)}

{step === 2 && method === "qris" && (
  <div>
    <img src={selectedMethod.qris_url} alt="QRIS" />
    <button onClick={() => setStep(3)}>Lanjut</button>
  </div>
)}

{/* Step 3: Upload File */}
{/* Step 4: Success/Error */}
</div>
);
}

```

Penjelasan:

- **Multi-step wizard** dengan state management (step 1-4)
- Progress indicator berubah warna dinamis sesuai progress
- **Conditional rendering** berdasarkan:
 - Current step
 - Payment method (rekening vs QRIS)
 - Success/error state
- Fetch payment methods dari database saat modal open
- Dynamic navigation: “Sebelumnya” & “Lanjut” buttons

Dynamic Behavior:

- Step 1: List payment methods dari database (bisa bertambah/berkurang)
 - Step 2: Display berbeda untuk rekening (text) vs QRIS (image)
 - Progress dots: gray → emerald (success) atau red (error)
 - Success modal: green checkmark icon
 - Error modal: red X icon dengan “Coba Lagi” button
-

4. Dynamic Room List dengan Filter

File: src/app/dashboard-user/page.tsx

Implementasi:

```
export default function DashboardUserPage() {
  const [rooms, setRooms] = useState<Room[]>([]);
  const [ksts, setKsts] = useState<Kost[]>([]);
  const [selectedKost, setSelectedKost] = useState<string | null>(null);
  const [showKostSelection, setShowKostSelection] = useState(false);

  useEffect(() => {
    const fetchData = async () => {
      // Fetch ksts berdasarkan referral code
      const response = await fetch('/api/get-ksts');
      const data = await response.json();
      const kstsData = data.ksts;

      setKsts(kstsData);

      // Dynamic flow berdasarkan jumlah ksts
      if (kstsData.length > 1 && !selectedKost) {
        setShowKostSelection(true); // Show selection UI
        return;
      }

      // Fetch rooms untuk ksts terpilih
      const kostId = selectedKost || kstsData[0].id;
      const { data: roomsData } = await supabase
        .from("rooms")
        .select("*")
        .eq("kost_id", kostId)
        .order("room_number", { ascending: true });

      setRooms(roomsData || []);
    };

    fetchData();
  }, [selectedKost]); // Re-fetch saat ksts berubah

  // Dynamic UI rendering
  if (showKostSelection) {
    return (
      <div className="grid gap-4">
        {ksts.map((kost) => (
          <button onClick={() => {
            setSelectedKost(kost.id);
            setShowKostSelection(false); // Hide selection
          }}>

```

```

        {kost.name}
      </button>
    )))
  </div>
);
}

return (
  <div>
    {/* Info banner jika multiple costs */}
    {selectedKost && costs.length > 1 && (
      <div className="bg-emerald-50 p-4">
        <p>Kost Terpilih: {costs.find(k => k.id ===
selectedKost)?.name}</p>
        <button onClick={() => {
          setSelectedKost(null);
          setShowKostSelection(true);
        }}>
          Ganti Kost
        </button>
      </div>
    )}

    <RoomList rooms={rooms} onSelect={setSelectedRoom} />
  </div>
);
}

```

Penjelasan:

- **Conditional UI** berdasarkan jumlah kost yang dimiliki owner
- Jika owner punya >1 kost: tampilkan kost selection
- Jika 1 kost atau sudah pilih: tampilkan room list
- Filter rooms dinamis berdasarkan selectedKost
- Info banner muncul hanya jika punya multiple costs dan sudah pilih

Dynamic Behavior:

- UI berubah total: selection cards → room grid
 - “Ganti Kost” button: kembalikan ke selection mode
 - Rooms di-filter real-time saat ganti kost
 - Empty state jika kost tidak punya rooms
-

5. Booking Form dengan Duration Calculator

File: src/app/dashboard-user/components/BookingForm.tsx

Implementasi:

```
export function BookingForm({ roomId, kostId, onSuccess }: BookingFormProps)
{
  const [duration, setDuration] = useState(1);
  const [state, formAction] = useActionState(createBookingRequest,
initialState);

  return (
    <form action={formAction}>
      <input type="hidden" name="room_id" value={roomId} />
      <input type="hidden" name="kost_id" value={kostId} />
      <input type="hidden" name="duration" value={duration.toString()}
/>

      <input
        type="number"
        min="1"
        max="36"
        value={duration}
        onChange={(e) => setDuration(Number(e.target.value))}
        placeholder="Masukkan durasi sewa dalam bulan"
      />

      <SubmitForm formState={state} onSuccess={onSuccess} />
    </form>
  );
}
```

Server Action (src/app/dashboard-user/actions.tsx):

```
export async function createBookingRequest(state: ServerActionState,
formData: FormData) {
  const duration = Number(formData.get('duration'));
  const dueDate = addMonths(new Date(), duration); // Dynamic calculation

  const { error } = await supabase.from('booking_requests').insert({
    user_id: user.id,
    room_id: roomId,
    kost_id: kostId,
    due_date: dueDate.toISOString(),
    status: 'pending',
  });

  return { success: !error, message: error ? error.message : 'Berhasil!' };
}
```

Penjelasan:

- User input duration (1-36 months)
- Client-side state management untuk input value
- Server-side calculation: `due_date = NOW() + duration months`
- Menggunakan `date-fns` library untuk date manipulation
- `useActionState` untuk integrate dengan server action

Dynamic Behavior:

- Input range: 1-36 (validation)
 - Due date calculated server-side berdasarkan duration
 - Success/error notification setelah submit
-

6. Dynamic Facility Selection

File: src/app/dashboard-admin/manage-rooms/add/components/RoomForm.tsx

Implementasi:

```
const predefinedFacilities = [
  { id: 'wifi', name: 'WiFi', icon: Wifi },
  { id: 'parkir', name: 'Tempat Parkir', icon: Car },
  { id: 'tv', name: 'TV', icon: Tv },
  // ... more facilities
];

export default function RoomForm({ kostId }: RoomFormProps) {
  const [formData, setFormData] = useState({
    facilities: [] as string[],
    customFacility: ''
  });

  const handleFacilityToggle = (facility: string) => {
    setFormData(prev => ({
      ...prev,
      facilities: prev.facilities.includes(facility)
        ? prev.facilities.filter(f => f !== facility) // Remove
        : [...prev.facilities, facility] // Add
    }));
  };

  const addCustomFacility = () => {
    if (formData.customFacility.trim()) {
      setFormData(prev => ({
        ...prev,
        facilities: [...prev.facilities, prev.customFacility.trim()],
        customFacility: '' // Reset input
      }));
    }
  };

  return (
    <form>
      {/* Predefined facilities - Dynamic selection */}
      <div className="grid grid-cols-3 gap-3">
        {predefinedFacilities.map((facility) => {
          const isSelected =
formData.facilities.includes(facility.name);
          return (
            <button
              type="button"
              onClick={() =>
handleFacilityToggle(facility.name)}
            >

```

```

        className={isSelected
          ? 'bg-emerald-50 border-emerald-500'
          : 'bg-gray-50 border-gray-200'
        }
      >
        <facility.icon className="w-4 h-4" />
        {facility.name}
        {isSelected && <CheckCircle2 className="ml-auto"
      />}
    </button>
  );
  }}}
</div>

{/* Custom facility input */}
<input
  value={formData.customFacility}
  onChange={(e) => setFormData(prev => ({
    ...prev,
    customFacility: e.target.value
  })))}
  onPress={(e) => e.key === 'Enter' && addCustomFacility()}
/>
<button type="button" onClick={addCustomFacility}>
  <Plus className="w-4 h-4" />
</button>

{/* Selected facilities chips - Dynamic List */}
<div className="flex flex-wrap gap-2">
  {formData.facilities.map((facility, index) => (
    <span key={index} className="bg-emerald-100 px-3 py-1
rounded-full">
      {facility}
      <button onClick={() =>
removeFacility(facility)}>x</button>
    </span>
  )
  )}
</div>
</form>
);
}

```

Penjelasan:

- Predefined facilities dengan icon mapping
- **Toggle selection:** click untuk add/remove
- **Custom facilities:** user bisa tambah facility sendiri
- Enter key support untuk quick add
- Selected facilities ditampilkan sebagai removable chips

- Dynamic styling: emerald (selected) vs gray (unselected)

Dynamic Behavior:

- Checkmark icon muncul saat selected
 - Background color berubah saat toggle
 - Chips list bertambah/berkurang sesuai selection
 - Custom facility langsung masuk ke selected list
-

7. Conditional Modals & Dialogs

File: src/app/dashboard-admin/manage-rooms/components/DeleteRoomButton.tsx

Implementasi:

```
export default function DeleteRoomButton({ roomId, roomNumber }: Props) {
  const [showModal, setShowModal] = useState(false);
  const [confirmText, setConfirmText] = useState("");
  const [error, setError] = useState("");
  const [showSuccessModal, setShowSuccessModal] = useState(false);

  const handleConfirm = async (e: React.FormEvent) => {
    e.preventDefault();

    if (confirmText !== "HAPUS") {
      setError("Teks konfirmasi tidak sesuai");
      return;
    }

    const result = await deleteRoom(roomId);

    if (result.success) {
      setShowModal(false);
      setShowSuccessModal(true); // Switch modals
    } else {
      setError(result.error);
    }
  };

  return (
    <>
      <button onClick={() => setShowModal(true)}>
        🗑 Hapus Kamar Permanen
      </button>

      {/* Confirmation Modal */}
      {typeof document !== "undefined" && showModal && createPortal(
        <div className="fixed inset-0 bg-black/40">
          <div className="bg-white rounded-xl p-6">
            <h3>Konfirmasi Penghapusan</h3>
            <p>Ketik "HAPUS" untuk mengkonfirmasi:</p>

            <input
              value={confirmText}
              onChange={(e) => {
                setConfirmText(e.target.value);
                setError(""); // Clear error saat typing
              }}
              onKeyDown={(e) => e.key === "Enter" && handleConfirm(e as any)}
            />
          </div>
        </div>
      )}
    </>
  );
}
```

```

    />

    {error && <p className="text-red-600">{error}</p>}

    <button onClick={handleCancel}>Batal</button>
    <button onClick={handleConfirm}>Hapus Kamar</button>
  </div>
</div>,
document.body
)}

{/* Success Modal */}
{typeof document !== "undefined" && showSuccessModal && createPortal(
  <div className="fixed inset-0 bg-black/40">
    <div className="bg-white rounded-xl p-6">
      <div className="bg-green-100 rounded">✅</div>
      <h3>Berhasil Dihapus</h3>
      <button onClick={handleSuccessClose}>OK</button>
    </div>
  </div>,
  document.body
)}
</>
);
}

```

Penjelasan:

- **Dual modals:** confirmation → success
- Portal rendering untuk proper z-index stacking
- Real-time validation: error muncul jika text !== "HAPUS"
- Error clear otomatis saat user typing
- Enter key support untuk submit
- Smooth transition antar modals

Dynamic Behavior:

- Modal muncul saat button click
 - Error message muncul/hilang dinamis
 - Switch dari confirmation ke success modal
 - Loading state saat deleting
 - Redirect after success
-